SELENIUM PYTHON AUTOMATION MODULE 3 LAB WORKBOOK

Python BDD Automation with Behave — Assignments 10, 11 and 12

Module	Module 3 — Python BDD / Behave Automation
Assignments	10 — Selenium Python + Behave BDD 11 — Data-Driven Automation in Behave 12 — Selenium Page Object Model in Behave
Application	SauceDemo web application
Primary tools	Python, Selenium WebDriver, Behave, Gherkin
Execution	Behave command-line runner

Module overview

The third module applies Python-based Behaviour-Driven Development (BDD) to web automation. The curriculum identifies three practical scenarios: setting up Selenium Python with Behave, performing test-data-driven automation in Behave, and implementing Selenium Page Object Model within the Behave framework. The completed assignments demonstrate this progression using Gherkin feature files, reusable step definitions, Selenium WebDriver and page objects.

Learning outcomes demonstrated

- Create and execute a basic BDD automation scenario using Behave and Gherkin.
- Use Scenario Outline and Examples to execute the same workflow with multiple data sets.
- Separate page interaction logic from BDD step definitions using the Page Object Model.
- Validate successful and unsuccessful login outcomes through automated assertions.
- Execute the Behave suites from the terminal and interpret feature, scenario and step results.

Experiment 1 — Assignment 10: Selenium Python + Behave BDD

Problem statement

Set up a Python BDD automation framework and automate an end-to-end login scenario. The test must open the SauceDemo login page, enter valid credentials, submit the login form, and verify that the inventory page is reached.

Objective

To establish the basic Behave framework structure and connect Gherkin business-readable steps with Selenium WebDriver implementations.

Tools and concepts

- Python and Selenium WebDriver with Microsoft Edge.
- Behave as the Python BDD test runner.
- Gherkin Feature and Scenario syntax.
- Step definitions using Given, When and Then.
- Element identification using ID, NAME and XPath.

Implementation

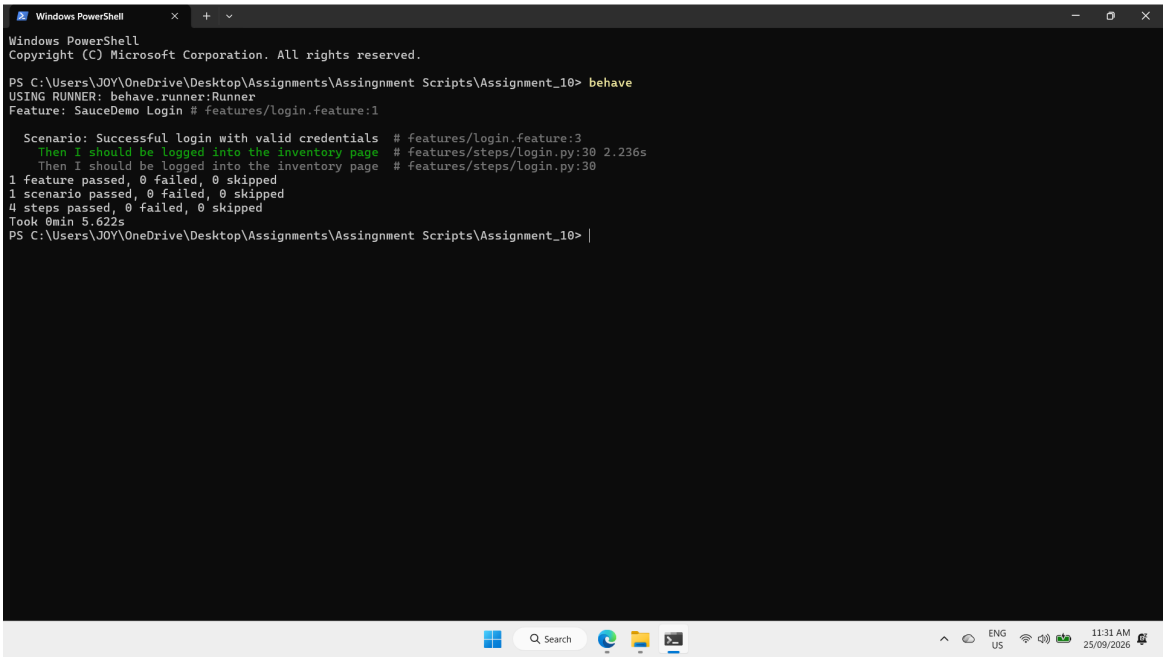
The feature file defines a single successful-login scenario. The step-definition file creates the browser, opens SauceDemo, enters standard_user and secret_sauce, clicks the login button, and verifies that /inventory.html is present in the current URL. The browser is closed after validation.

Execution command

```
behave
```

Execution evidence

Behave BDD Execution Output



Observed result: 1 feature passed, 1 scenario passed and 4 steps passed; no failures or skipped steps were reported.

Experiment 2 — Assignment 11: Data-Driven Automation in Behave

Problem statement

Extend the basic Behave login automation so that the same scenario can be executed with multiple username/password combinations and can validate both successful and unsuccessful outcomes.

Objective

To implement test-data-driven automation in Behave using Gherkin Scenario Outline and Examples, while keeping the step definitions reusable for every data set.

Tools and concepts

- Behave BDD framework and Gherkin Scenario Outline.
- Examples table for externalized test cases within the feature.
- Parameterized step definitions receiving username, password and expected result.
- Selenium WebDriver for browser interaction.
- Validation of both inventory-page success and visible login-error states.

Test data implemented

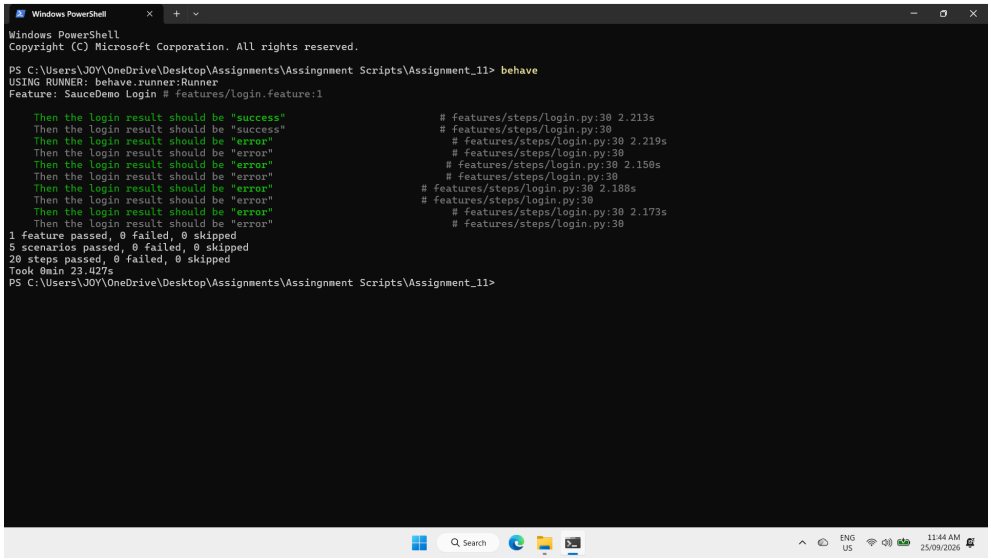
Username	Password	Expected
standard_user	secret_sauce	success
locked_out_user	secret_sauce	error
problem_user	wrong_password	error
wrong_user	secret_sauce	error
standard_user	wrong_password	error

Implementation and execution

The Scenario Outline passes each Examples row into the reusable login steps. For the success case, the test checks the inventory URL. For error cases, it locates the SauceDemo validation message and checks that it is displayed. The browser is closed after each scenario.

Execution evidence

Data-Driven Behave Execution Output



Observed result: 1 feature passed, 5 scenarios passed and 20 steps passed; no failures or skipped steps were reported.

Experiment 3 — Assignment 12: Selenium POM in the Behave Framework

Problem statement

Restructure the data-driven Behave login automation using the Page Object Model so that page locators and UI actions are separated from the Gherkin step definitions and test validation.

Objective

To apply POM within Behave by creating a reusable LoginPage class and using that class from the step-definition layer.

Architecture

Layer	Responsibility
login.feature	Defines the BDD feature and Scenario Outline in Gherkin.
steps/login.py	Maps Given/When/Then steps to automation workflow and assertions.
pages/login_page.py	Contains locators, page actions and page-state helper methods.
WebDriver	Provides browser control and element interaction.

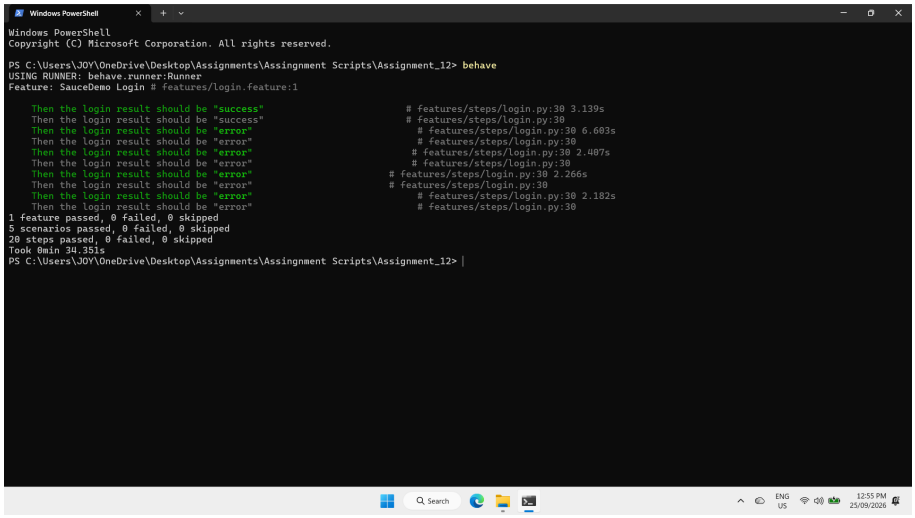
Page Object implementation

- LoginPage stores the WebDriver instance.
- enter_username() and enter_password() perform field interactions.
- click_login() performs the login action.
- is_login_successful() validates the inventory URL.
- is_error_displayed() validates the displayed error message.

The step definitions therefore use methods such as enter_username(), enter_password() and click_login() instead of directly locating page elements. This separates page-specific implementation from the BDD workflow.

Execution evidence

POM Behave Execution Output



Observed result: 1 feature passed, 5 scenarios passed and 20 steps passed; no failures or skipped steps were reported.

Module 3 — Overall Result and Learning Outcome

Consolidated execution results

Assignment	Framework focus	Result
10	Selenium Python + Behave BDD	1 feature / 1 scenario / 4 steps passed
11	Data-driven Behave automation	1 feature / 5 scenarios / 20 steps passed
12	POM in Behave	1 feature / 5 scenarios / 20 steps passed

Key observations

• Behave provides a readable BDD layer in which business behaviour is expressed through Gherkin features and scenarios.

• Step definitions connect Gherkin language with executable Selenium Python actions.

• Scenario Outline and Examples allow one reusable workflow to cover multiple login data combinations.

• POM reduces duplication by centralizing locators and page-level actions in a dedicated page class.

• The three assignments demonstrate a progression from basic BDD execution to data-driven automation and framework-level organization.

Result

All three Module 3 assignments were executed successfully. The terminal evidence confirms that the Behave features, scenarios and steps completed without failures or skipped tests. The implementations demonstrate the required Selenium Python, Behave BDD, data-driven and Page Object Model concepts specified for the module.

Module-level conclusion

Module 3 established a practical Python BDD automation workflow using Selenium WebDriver and Behave. The work progressed from a basic end-to-end BDD scenario, through reusable test-data-driven execution, to a Page Object Model architecture. This provides a structured foundation for maintainable automation frameworks using Gherkin, reusable step definitions and page objects.

Reference

Course curriculum: CoE-TOC-PYTHON_AUTOMATION.xlsx, Python Automation ToC — Module 3 / Assignments 1–3 (Selenium Python + Behave BDD, data-driven Behave automation, and Selenium POM in Behave).

Execution evidence: labelled screenshots from Assignments 10, 11 and 12.

Implementation evidence: completed feature files, Behave step definitions and Page Object Model source files used for the module.